Automated testing documentation

Student Planner (Kivy/SQLite)

Desktop Productivity Application
Software Engineering
June 8, 2026

I. Test Environment Metrics

The tests were conducted in a heterogeneous environment, which allowed for the verification of the application's stability across different platforms and interpreter versions.

Environment A (main environment): macOS 15.7 (Apple Silicon ARM64), Python 3.9.6, Kivy 2.3.1

Environment B (Linux Environment): Linux (Ubuntu/Generic x86_64), Python 3.13.12

Testing Framework: pytest (versions 8.4.2 - 9.0.3)

II. Purpose and Scope of Tests

The main goal of the QA process was to ensure the reliability of key system modules before the production release. The scope of integration and unit tests included:

Database module: create, read, update operations, cascading deletes, state management.

Forms module (adding events): input validation (text data, dates, times, negative values), generating error messages.

Views module (calendar): day rendering logic, event filtering, view state management.

Progress module (progress tracker): mathematical progress calculations, negative value blocks.

Special attention was given to edge case testing, including incorrect date and time formats, negative values, very large numeric values, empty form fields, long text strings, and data containing Polish characters and Unicode symbols.

III. Iteration and Defect Resolution History

The testing process took an iterative form. In the early phases, validation and configuration errors were identified, which were gradually eliminated in subsequent phases.

1. Module: Adding a Course (test_add_subject.py)

Initial phase (May 30, 2026): 43 tests were executed. 16 inconsistencies were detected.

Defect description: The application reacted incorrectly to extreme input data. The validator allowed an empty instructor field, negative values (e.g., negative duration, negative points), and incorrect date formats (e.g., "abc", "2027/01/30"). Instead of blocking the save and displaying an error window (show_error_popup), the system allowed the faulty data to be added to the database.

Final phase (May 31, 2026): Fixes were implemented in the form logic layer. An additional test was added. 44 tests were executed. Result: 100% Passed.

2. Module: Exams and Colloquiums (test_exams_colloquiums.py)

Initial phase (May 29, 2026): 32 tests were executed. 4 integration errors were detected.

Defect description: Problems appeared during the testing of incorrect date validations (e.g., "2026-05-32", "2026-13-10", "2026-00-15", "2026-05-00"). I Instead of correctly rejecting the wrong data and displaying a validation message, the application reported a Kivy framework exception. The analysis showed that the problem was related to both the handling of validation cases and the incomplete initialization of Kivy graphical components in the test environment. As a result, some tests could not be executed correctly.

Final phase (May 31, 2026): Fixes were introduced in the date validation logic, and the test environment configuration for the Kivy framework was adjusted. Additionally, the test suite was expanded with stricter verification of date and time formats (e.g., "23:60", "25:00", "-1:30", "YYYY-MM-DD", "2026-5-5"). A total of 44 tests were executed. Final result: 100% Passed.

3. Module: Calendar (test_calendar.py)

Verification phase (May 31, 2026): 21 tests were executed on the Linux/Python 3.13 environment. 2 test environment errors were detected.

The anomalies identified in the logs concerned only the unit test isolation layer. The business logic of the application in each of these cases worked according to the specification. The above corrections allowed for a full, positive test result (100% Passed).

4. Stable Modules (Database and Progress Tracker)

These modules showed 100% effectiveness from the first iteration.

Database (test_database_function.py): 30 tests. Limits (CHECK constraints), cascading deletion of relations, and the logic of saving unique notes were tested. Result: 100% Passed.

Tracker (test_progress.py): 5 tests. Verification of text formatting and negative point blocks passed flawlessly. Result: 100% Passed.

IV. Defect Summary and Technical Notes

Most of the problems concerned data validation in forms and test environment configuration.

The highest number of errors was detected in the adding courses module. Tests showed that the application accepted some invalid input data, such as negative values, wrong date formats, or some incomplete form data. After introducing additional validation mechanisms, all detected problems were removed, and the module achieved a full positive result.

In the exams and colloquiums module, problems were initially identified during the testing of incorrect dates and times. In selected cases, instead of correctly handling the wrong data, the application reported a Kivy framework exception. The analysis showed that the problems were related to both handling validation cases and test environment configuration. In subsequent iterations, validation mechanisms were refined, and the test configuration was adjusted, while expanding the set of test cases with additional incorrect date and time formats. After the fixes were implemented, all module tests ended with a positive result.

During the testing of the calendar module, two failed test cases were noted. The first resulted from incorrect preparation of the test object responsible for checking the calendar refresh method call. The second was related to the lack of application object initialization in the test environment. In both cases, the problem concerned only the tests themselves, not the application's operation. The logic of the calendar module worked according to the project assumptions.

The database and progress tracking modules did not show any errors during the entire testing process. All test cases ended with a positive result right in the first iteration.

———————————————————————————————————————

UI consistency notes:

During the final phase of manual testing on different operating systems, differences in rendering interface elements were noted. The application shows high logical stability; however, in the macOS environment, subtle discrepancies in the positioning and formatting of some graphical elements were observed compared to Linux and Windows environments.

The identified anomalies include:

Notifications: The notification button (bell icon) shows slight coordinate shifts, resulting in a lack of perfect centering relative to the top bar on macOS.

Calendar: The calendar formatting mechanism shows minimal differences in padding and spacing, which affects the aesthetics of the view but does not disrupt the functionality or readability of the schedule.

The identified issues are purely cosmetic and result from differences in how the graphics engine interprets metrics (dp) across different operating systems. The business logic and data correctness remain fully consistent with the design assumptions. In future versions, it is possible to introduce dedicated style profiles to achieve identical visual representation across all supported platforms.

V. Summary

The conducted testing process allowed for the detection and removal of all significant problems identified during the application's development. The verification included both unit and integration tests executed automatically using the pytest framework, as well as manual tests consisting of checking the operation of individual functions from the end-user's perspective.

After introducing the necessary fixes, all system modules described in this report achieved a result of 100% successfully completed automated tests. Additionally, the conducted manual tests did not reveal any errors preventing the use of the application or problems affecting the correct operation of its key functions.

The obtained results confirm the stability, correctness, and operational consistency of the Student Planner application. Based on the conducted tests, it can be stated that the system meets the design assumptions and is ready for use.